

Multi-Tenant URL Shortener & Analytics Platform

A full-stack SaaS platform for tenant-isolated link management, click analytics, and role-based access control

Stack: NestJS · React · PostgreSQL · Redis · Prisma · TypeScript **Architecture:** REST API Monorepo (npm Workspaces)

Deployment: Docker-Compose (Postgres 16 + Redis 7)

1. ABSTRACT

This project delivers a production-grade multi-tenant URL shortening and analytics platform. Organisations (tenants) are provisioned as isolated namespaces on shared infrastructure, each managing their own short links, users, and click telemetry. The system exposes two distinct traffic paths: an **authenticated management API** (link CRUD, analytics dashboards, user management) protected by a three-stage NestJS guard chain, and a **public redirect path** optimised for minimal latency via a Redis cache-aside strategy. Role-based access control (RBAC) distinguishes three roles — MEMBER, TENANT_ADMIN, SUPER_ADMIN — enforced through a permission-based guard and a Prisma Client Extension that transparently injects tenant scoping into every database query. The React frontend presents tenant-scoped dashboards and a platform-wide admin console. The goal is correct data isolation, high redirect throughput, and an extensible architecture ready for horizontal scale.

2. PROBLEM STATEMENT

SaaS teams building URL shorteners face two competing concerns: *shared infrastructure cost* (multiple tenants on one deployment) and *strict data isolation* (tenant A must never see tenant B's links or analytics). Existing open-source shorteners are single-tenant by design; retrofitting isolation via application-level filters is error-prone because a single missed `WHERE tenantId = ?` clause causes a cross-tenant data leak. Additionally, the redirect endpoint is the highest-traffic route and must not be gated behind authentication overhead, yet click attribution must still be correctly stamped per tenant.

3. OBJECTIVES

- Tenant isolation by construction:** make it structurally impossible to write an unscoped query against tenant-owned tables by centralising filtering in a Prisma Client Extension.
- Latency-optimised redirects:** serve short-link redirections from Redis (cache-aside, TTL bounded by link expiry) so Postgres is touched only on first click or cache eviction.
- Granular RBAC:** implement a permission-based guard covering URL creation, management, analytics, and platform-level admin operations.
- Secure token lifecycle:** issue short-lived JWT access tokens (15 min) and 7-day rotating refresh tokens; hash refresh tokens with SHA-256 using `timingSafeEqual` comparison.
- Observability foundation:** structured JSON logging (nestjs-pino), Redis-backed rate limiting, and a dedicated health endpoint to support future production deployment.

4. EXISTING VS. PROPOSED SYSTEM

⚠️ EXISTING LIMITATIONS

- Single-tenant URL shorteners (e.g. YOURLS, Polr) with no isolation model
- Application-level filtering — every developer must remember `WHERE tenantId` at every call site
- Redirect and management traffic share the same pipeline, adding unnecessary auth overhead
- No standardised RBAC; permissions hardcoded per endpoint or absent entirely

✓ PROPOSED SYSTEM

- Tenant scoping enforced by Prisma Client Extension — impossible to write an unscoped query using `TenantPrismaService`
- Redirect path registered on raw Express before NestJS router; bypasses all guard overhead entirely
- Declarative permissions (`@RequirePermissions()`) with a centralised permission-to-role mapping table
- Rotating refresh tokens with hash stored per user row; logout invalidates the session immediately

5. SYSTEM ARCHITECTURE

FIG. 1 — SYSTEM ARCHITECTURE: TWO DISTINCT TRAFFIC PATHS ON SHARED INFRASTRUCTURE

PUBLIC REDIRECT PATH (UNAUTHENTICATED)

Visitor clicks short link — GET /:shortCode



Express Handler (redirect.handler.ts)
Registered before NestJS router



Redis Cache Lookup (redirect:<code>)



Cache HIT
302 Redirect

Cache MISS
Postgres lookup



Click logged async (non-blocking after 302)
tenantId stamped from URL row

AUTHENTICATED MANAGEMENT PATH

Tenant Admin / Super Admin — React Dashboard (Vite, port 5173)



NestJS Backend (port 3000) — Helmet · CORS · /api/v1



① JWT Auth Guard — Verifies token; attaches sub, tenantId, role



② RBAC Authorization Guard — @Roles / @RequirePermissions



③ TenantContext Interceptor — AsyncLocalStorage



Controllers → Services (Auth, URL, Analytics, Admin, Tenant)



TenantPrismaService (Prisma Client Extension)
Auto-injects tenantId into every query clause



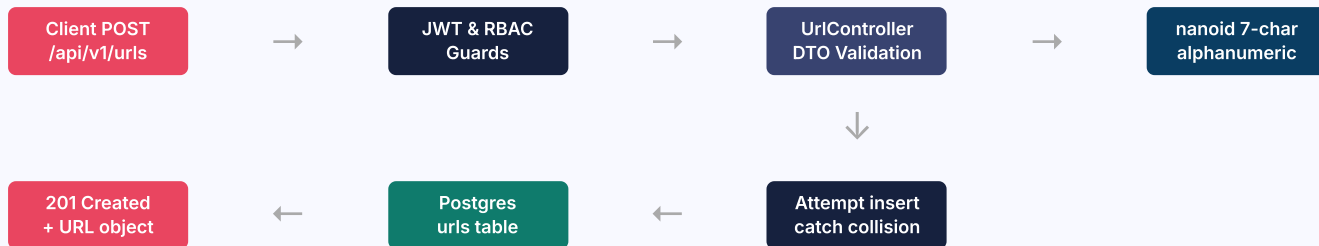
PostgreSQL 16 — tenants · users · urls · clicks

Shared Data Layer: Redis 7 — redirect cache (TTL ≤ 24h, bounded by link expiresAt), Redis-backed rate limiting (IP/tenant), refresh-token session cache. Cache invalidated unconditionally on URL update/delete. Redis is never the source of truth; Postgres is.

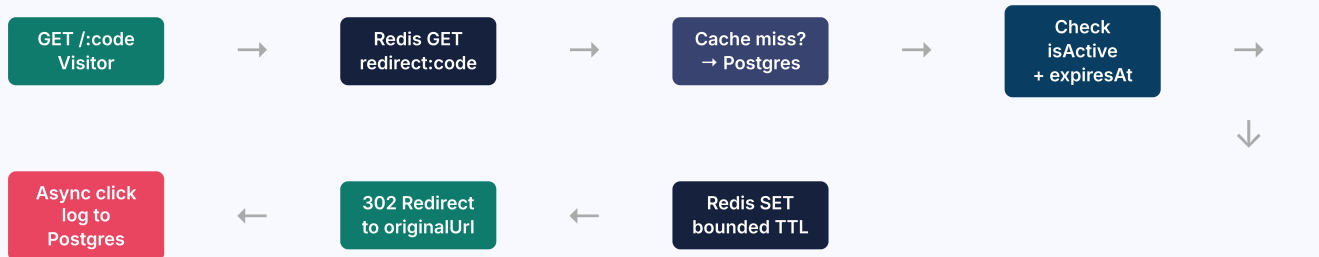
6. METHODOLOGY & TECHNICAL PIPELINE

FIG. 2 — TECHNICAL PIPELINE: URL CREATION FLOW AND REDIRECT FLOW

URL CREATION FLOW



REDIRECT FLOW



7. BACKEND MODULE ARCHITECTURE & RBAC DESIGN

AuthModule	UrIModule	Analytics
POST /auth/login	POST /urls	GET /analytics/overview
POST /auth/refresh	GET /urls (paginated)	GET /analytics/clicks
POST /auth/logout	GET /urls/:id	GET /analytics/referrers
POST /auth/register	PATCH /urls/:id	GET /analytics/user-agents
bcrypt cost=12	DELETE /urls/:id (soft)	Tenant-scoped queries
SHA-256 refresh hash	nanoid short codes	Browser/OS parsing

RedirectModule	AdminModule (SUPER_ADMIN only)	
Express handler (pre-Nest)	GET /admin/stats	GET /admin/tenants
GET /:shortCode	POST /admin/tenants	GET /admin/users
RedirectCacheService	GET /admin/links	PATCH /admin/links/:id
RESERVED_PATHS bypass	GET /admin/analytics/top-links	GET /admin/analytics/tenant-activity

RBAC PERMISSION MATRIX

Permission	MEMBER	TENANT_ADMIN	SUPER_ADMIN
url:read	✓	✓	✓
url:create	✓	✓	✓
url:manage_own	✓	✓	✓
url:manage (any)	—	✓	✓
analytics:read	✓	✓	✓
tenant:manage	—	✓	✓
platform:read/manage	—	—	✓

TENANT ISOLATION MECHANISM

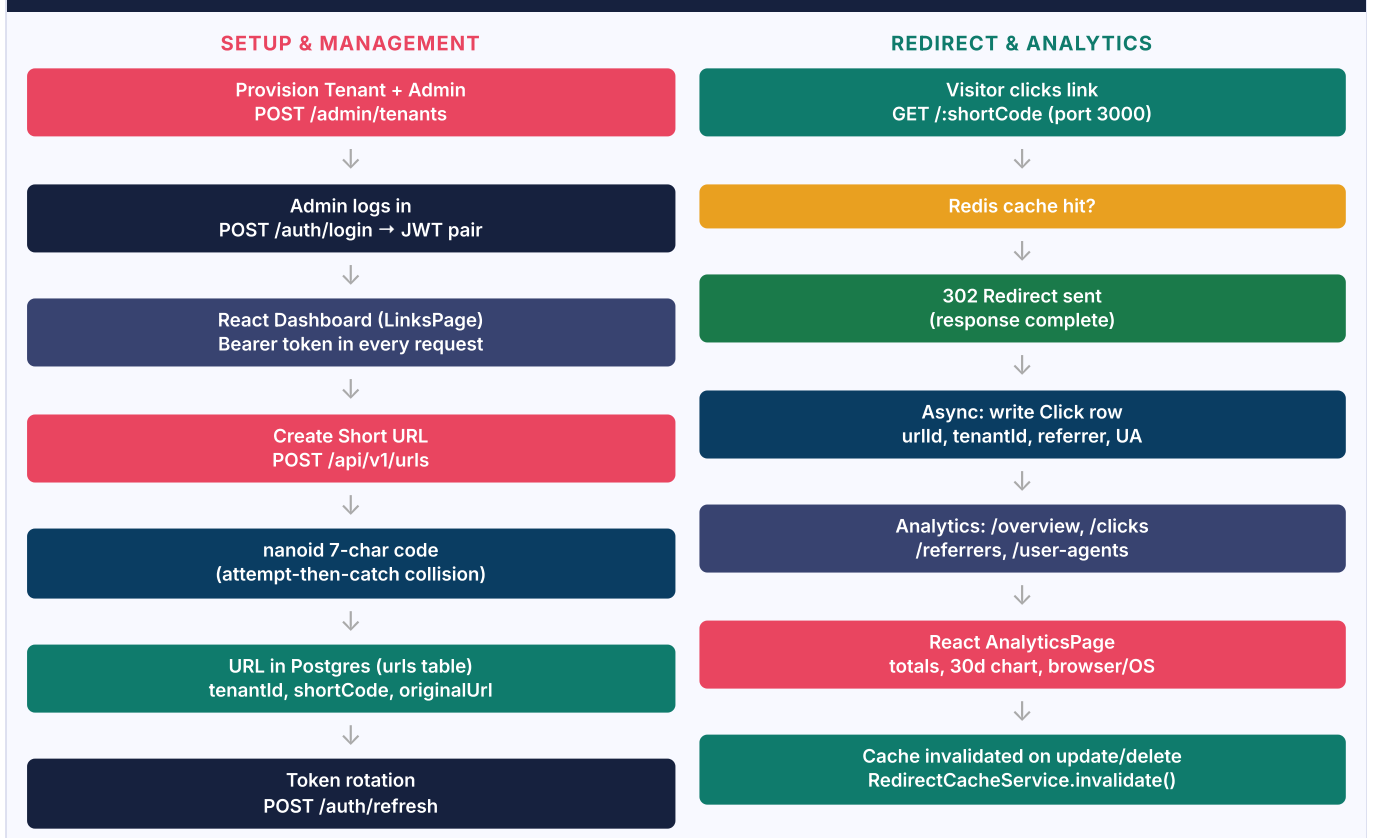
- **TenantPrismaService** wraps Prisma via Client Extension, merging `tenantId` into every `where` / `data` clause at query time.
- **AsyncLocalStorage** propagates `tenantId` from JWT through the entire async call chain — no argument threading required.
- **Fail-closed:** if no tenant context is active, the extension throws `ForbiddenException` rather than running an unscoped query.
- **Exceptions:** login + redirect use raw `PrismaService` (globally unique email / shortCode; no caller identity available).

8. TECHNOLOGY STACK

Category	Technology / Version
Language	TypeScript 5.7 (strict mode, monorepo)
Backend Framework	NestJS 11 (Express adapter)
Frontend	React 18.3 + Vite 6 + React Router 7
Database	PostgreSQL 16 (Docker-managed)
ORM	Prisma 6.3 (Client Extension for tenant scoping)
Cache / Rate Limit	Redis 7 (ioredis 5.4)
Authentication	JWT (@nestjs/jwt); bcrypt 6; SHA-256 refresh hash
Short Code Gen	nanoid 3 (custom alphanumeric alphabet, 7 chars)
HTTP Security	Helmet 8; CORS; ValidationPipe (class-validator)
Logging	nestjs-pino (structured JSON); pino-pretty (dev)
Testing	Jest 29 + ts-jest (url.service.spec, analytics.service.spec)
Infrastructure	Docker Compose; npm Workspaces monorepo

9. COMPLETE END-TO-END WORKFLOW

FIG. 3 — COMPLETE END-TO-END WORKFLOW



10. KEY DESIGN DECISIONS & VERIFIED BEHAVIOURS

Redirect Correctness Verified

Link created with 3-second `expiresAt` showed matching Redis TTL; returned 302 before expiry, 410 Gone immediately after — cache expiry and Postgres enforcement aligned without re-checking on every cache hit.

Tenant Isolation Verified

Two seeded tenants tested: `findUnique` via `TenantPrismaService` for another tenant's real user ID returns `null` — cross-tenant rows inaccessible even with a valid ID.

Timing Attack Prevention

Login runs bcrypt comparison even for unknown emails (against a fixed decoy hash), eliminating the timing difference between "no such user" and "wrong password" responses.

11. CONCLUSION & FUTURE SCOPE

The platform achieves its core goals: correct tenant isolation enforced at the ORM layer, a latency-optimised public redirect path, declarative RBAC, secure token rotation, and a React dashboard covering link management and per-tenant click analytics. The deliberate separation of the redirect and management code paths allows each to be optimised independently without compromising the other.

DEFERRED / FUTURE IMPROVEMENTS

- **Async click recording:** queue-backed write (BullMQ) to decouple redirect latency from DB write latency.
- **Immediate access-token revocation:** Redis denylist to close the 15-minute post-logout window.
- **Analytics at scale:** dedicated time-series store or event stream when click volume exceeds relational comfort.
- **Edge / Nginx layer:** TLS termination and edge-level rate limiting for production deployment.
- **Schema-per-tenant:** justified only when contractual data residency or dedicated SLAs are required.